

Spring Boot Internals & Auto-Configuration

1. Walk me through the internal startup process of a Spring Boot application.

The startup process begins when the main method calls **SpringApplication.run()**. This method first creates an **ApplicationContext** instance and configures the environment. Next, the **Auto-Configuration** process runs, followed by **Component Scanning** to find user-defined beans. Finally, the context is refreshed, all necessary beans are initialized and wired, and the application starts the embedded server.

2. How does Spring Boot resolve conflicting versions when different starters pull different dependencies?

Spring Boot resolves version conflicts using the **Bill of Materials (BOM)** approach, inherited via the **spring-boot-starter-parent**. The parent POM defines a central, compatible set of version numbers for all key dependencies. If different starters request differing versions of a library (like Hibernate), Spring Boot aligns them to the single, centrally managed version to prevent conflicts.

3. How do you disable a specific auto-configuration class in advanced scenarios?

You can disable a specific auto-configuration class by adding it to the exclusion list. This is typically done by setting the property **spring.autoconfigure.exclude** in your **application.properties** or **application.yml** file, specifying the fully qualified class name of the configuration you wish to turn off.

4. How does auto-configuration actually contribute to the bootstrapping of the application?

Auto-configuration contributes by executing code that creates default beans based on conditions met in the classpath. It essentially runs predefined **@Configuration** classes that set up common components (like a database connection, web configuration, or logging factory) automatically, allowing the application to immediately run without manual setup.

5. Explain how to customize a specific auto-configuration at a fine-grained level.

You customize auto-configuration primarily through external configuration properties. Each auto-configuration class binds to specific keys (e.g., `server.port` or `spring.datasource.url`). By setting these values in your `application.properties`, you change the default behavior or values used when Spring creates the corresponding bean.

6. What are conditional annotations, and how do they influence the auto-config pipeline?

Conditional annotations (like `@ConditionalOnClass` or `@ConditionalOnMissingBean`) are boolean expressions that control whether a configuration class or bean definition should be processed. They influence the pipeline by allowing auto-configuration to run **only** if the necessary external libraries are present or if the required bean has **not** already been created by the user.

7. What specific role does `@EnableAutoConfiguration` play under the hood?

The specific role of `@EnableAutoConfiguration` is to trigger the entire auto-configuration mechanism. It instructs Spring to search for all JAR files containing a **META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports** file, which lists all the potential auto-configuration classes that the framework should consider loading.

8. If multiple embedded servers are present in the classpath, how does Spring Boot choose which one to start?

Spring Boot uses an **ordered exclusion system** to choose the server. It checks for specific server classes (Tomcat, Jetty, Undertow) and typically prioritizes Tomcat if multiple are found. The framework will try to start the first server it finds in its internal order unless other servers are explicitly excluded or included.

9. How can you disable the default server and activate another embedded server?

You disable the default server (e.g., Tomcat) by adding an **@Configuration** class and defining a custom bean that overrides the default server factory, or by using the `<exclusion>` tag in your

build file. You activate a different server by ensuring its starter dependency (e.g., spring-boot-starter-jetty) is present in the classpath.

10. How does Spring Boot simplify dependency injection compared to traditional Spring XML projects?

Spring Boot simplifies DI by enabling **Component Scanning** and **Auto-wiring** by default. Traditional Spring required developers to manually list every bean in an XML file and explicitly define their dependencies. Spring Boot automatically finds beans using annotations (like `@Service`) and injects dependencies using `@Autowired`, eliminating the XML configuration overhead.

11. How do you force an auto-configuration to “back off” when a bean already exists?

Auto-configuration is designed to "back off" by default when a bean already exists. This behavior is enforced by the `@ConditionalOnMissingBean` annotation within the auto-configuration class. To leverage this, simply define your custom bean with the correct type, and Spring will see it and skip the default auto-configuration process.

Here are the answers to your questions regarding microservices architecture and implementation with Spring Boot.

Service Communication & Management

Q12. How would you implement service-to-service communication in a microservices system built with Spring Boot?

A: I would implement service-to-service communication using two main styles. For synchronous requests, I would use a REST client like **RestTemplate** or **WebClient** (for reactive services). For asynchronous communication, I would use a **Message Broker** (like Kafka or RabbitMQ). This design ensures low coupling and high throughput.

Q13. What is Spring Cloud and why is it relevant for building distributed systems?

A: **Spring Cloud** is a set of tools and libraries built on top of Spring Boot specifically designed to solve common problems in distributed systems. It is relevant because it provides ready-to-use patterns for service discovery, centralized configuration management, intelligent routing, and resilience mechanisms, which are difficult to implement from scratch.

Q14. How do you externalize and secure configuration in a microservices architecture?

A: I would externalize configuration using a central service like **Spring Cloud Config Server**. This server pulls configuration from a Git repository. To secure the configuration, I would encrypt sensitive values (like database passwords) before storing them in Git, and the Config Server would decrypt them only when pulling the values into the application.

Q15. How do you manage sensitive configuration values across multiple environments?

A: Sensitive values are managed using **encryption** or a dedicated **Secret Management** tool like HashiCorp Vault. These tools provide a secure, centralized location to store credentials. The microservice retrieves the secrets from the Vault at startup using a unique token, ensuring the sensitive data is never stored directly in the configuration files or Git.

Q16. Describe how you implement resilience patterns (retry, circuit breaker, fallback).

A: I implement resilience patterns using a library like **Resilience4J**. The **Retry** pattern automatically attempts a failed network call a few times. The **Circuit Breaker** pattern monitors failures and, if the failure rate is too high, it "opens the circuit" and immediately fails subsequent requests. The **Fallback** pattern executes predefined backup logic when the main service call fails, ensuring graceful degradation.

Security & Session Management

Q17. How do you secure microservices using Spring Security (gateway, shared token, user roles)?

A: Security is managed at the **API Gateway**, which handles user login and issues a **JWT (JSON Web Token)**. This token is passed to all downstream services. Spring Security on each service validates the token and extracts the **user roles and permissions**, applying security rules at the resource level.

Q18. How do you implement JWT-based authentication in distributed systems?

A: The **Authorization Server** (or API Gateway) authenticates the user and generates a cryptographically signed **JWT**. This token contains user information and permissions. The client includes this token in the Authorization header of every request. Each microservice then uses Spring Security to validate the signature and trust the user details inside the token.

Q19. What is method-level security and how do you apply it in service classes?

A: **Method-level security** controls access to specific Java methods, rather than the entire service endpoint. You apply it by using Spring Security annotations directly on the service methods, such as **@PreAuthorize("hasRole('ADMIN')")** or **@PostAuthorize**. This ensures the user's role is checked before the business logic is allowed to execute.

Q20. How do you handle session management across multiple microservice instances?

A: You avoid traditional server-side sessions entirely. Instead, you use **stateless authentication**, typically with **JWTs** or tokens. The JWT contains all necessary user information, eliminating the need to store session data on the server itself.

Q21. How do you configure session clustering or shared session stores in Spring Boot?

A: If sessions must be maintained, you configure a **shared session store** using Spring Session with a backing service like **Redis** or a distributed database. This ensures that session data is stored centrally, allowing any instance of the microservice to access the session, even if the request hits a different server.

Q22. Your application loses sessions when scaled horizontally, what strategy fixes this?

A: This is fixed by ensuring the application is **stateless** and using a **centralized session store**. When scaled horizontally, sessions should be moved out of the local memory of the individual server instance and into an external, shared store like **Redis**. This allows any new instance to retrieve the session data.

Caching & Performance Engineering

Q.23. How do you implement caching in a Spring Boot application?

A: You implement caching by enabling Spring's caching abstraction using the **@EnableCaching** annotation on a configuration class. Then, you apply caching annotations directly to your service methods, such as **@Cacheable("users")** on a read method or **@CachePut("users")** on an update method. This handles the cache interaction automatically.

Q.24. What caching providers are supported, and how do you choose one?

A: Spring supports various providers like **Redis**, **Ehcache**, **Caffeine**, and **Hazelcast**. You choose a provider based on your application's needs: use **Redis** for distributed caches (shared across microservices), and use **Caffeine** for fast, high-performance local caches.

Q.25. Explain the difference between cache eviction and cache expiration.

A: **Cache expiration** is time-based; an entry is automatically removed after a set amount of time (Time-To-Live or TTL), regardless of usage. **Cache eviction** is policy-based (e.g., Least Recently Used or LRU); the cache removes entries when it runs out of space, prioritizing items that haven't been accessed recently.

Q.26. How do you design caching for an application with extremely high read volume?

A: I would implement a multi-layered caching strategy: use a very fast **local cache (Caffeine)** on the application server to handle the highest read throughput instantly. For cache misses, I would fall back to a larger, shared **distributed cache (Redis)** before hitting the database.

Q.27. What limitations exist in Spring's default caching abstraction?

A: The main limitation is that the abstraction only handles basic **synchronous** operations. It doesn't natively handle advanced features like asynchronous caching or reactive stream integration. Also, implementing advanced policies like **cache-aside** often requires dropping down to the native provider API instead of just using the Spring annotations.

Q.28. Your app slows down under load. Walk through your performance diagnosis approach.

A: I start by checking system metrics like CPU usage and database connection pool health. If the CPU is high, I take a **thread dump (jstack)** to see where threads are spending their time (e.g., waiting for a lock or doing heavy calculation). I then use a profiler (like JFR) to identify **hotspots** and slow database queries that need optimization.

Q.29. What strategies do you use to improve Spring Boot application performance?

A: Strategies include implementing **caching** to reduce database load, tuning the **JVM heap size** and garbage collector, and optimizing resource-heavy database **SQL queries** by adding indexes. For I/O heavy apps, I might switch to **Spring WebFlux** to use non-blocking I/O.

Q.30. How do you make Spring Boot microservices more resilient under failure conditions?

A: I use **resilience patterns** from libraries like Resilience4J. I implement the **Circuit Breaker** pattern to stop calling failing services and immediately return a fallback response. I also use **Timeouts** and **Retries** on service calls to quickly recover from temporary network issues or slow responses.

Spring Security & Actuator

Q31. How do you secure Actuator endpoints in production?

A: You secure Actuator endpoints by integrating them with **Spring Security**. By default, most sensitive endpoints are restricted to local access only. In production, you expose them only over a separate, secure port and require specific user roles (like ADMIN) to access them using basic or token-based authentication.

Q32. What are Actuator endpoints and how do you customize them?

A: Actuator endpoints are internal HTTP URLs that expose monitoring and operational information about a running application, such as `/health`, `/metrics`, or `/env`. You customize them by configuring properties prefixed with **management.endpoint** to enable or disable specific endpoints, or by defining custom endpoints using the **@Endpoint** annotation.

Q33. Explain authentication vs authorization in Spring Security.

A: Authentication is the process of verifying who a user is (e.g., checking a username and password). **Authorization** is the process of verifying what an authenticated user is *allowed* to do (e.g., checking if a user has the ADMIN role to access a specific page). Authentication is "who are you?"; Authorization is "what can you do?".

Q34. How is Spring Security integrated into a Spring Boot application?

A: Spring Security is integrated by including the **spring-boot-starter-security** dependency. Spring Boot automatically detects this and applies basic security defaults, such as requiring authentication for every request and creating a default login form. You then customize the security rules by extending **SecurityFilterChain** (or **WebSecurityConfigurerAdapter** in older versions).

Q35. How would you configure form-based login in Spring Security?

A: You configure form-based login by overriding the **SecurityFilterChain** bean. Inside the configuration, you use the method **.formLogin()** to enable the default login form provided by Spring Security and specify where the login form is located and which URL should handle the submission.

Q36. How do you enforce security rules across microservices?

A: Security rules are enforced using an **API Gateway** which handles user authentication and issues a **JWT** (JSON Web Token). Downstream microservices then use Spring Security to validate the incoming JWT. Each service applies its own **method-level security** rules based on the user's roles found inside the token.

Q37. Explain how to secure a system using JWT end-to-end.

A: First, the client sends credentials to an authentication service which returns a signed **JWT**. The client then sends this JWT in the **Authorization header** of all subsequent requests. The microservice validates the JWT's signature against a known secret key and trusts the user details contained within the payload.

Q38. How do you secure sensitive data in multi-user Spring Boot applications?

A: Sensitive data is secured by applying **method-level security** (using `@PreAuthorize` or `@Secured`) to ensure only authorized users access certain service methods. Database credentials and API keys should be stored externally using **Secret Management** tools (like Vault) rather than in configuration files.

Q39. How do you configure session timeouts, CSRF, and logout handling?

A: These are configured within the **SecurityFilterChain** bean. **Session timeouts** are set using properties in `application.properties` (e.g., `server.servlet.session.timeout`). **CSRF protection** is enabled by default and customized using the `.csrf()` method. **Logout handling** is customized using the `.logout()` method, specifying the logout URL and where to redirect afterward.

Database, Transactions & Migrations**Q.40. How does Spring Boot handle database migrations (Flyway, Liquibase)?**

A: Spring Boot simplifies database migrations by providing auto-configuration for tools like **Flyway** and **Liquibase**. Both tools track schema changes in dedicated scripts (SQL for Flyway, XML/YAML/SQL for Liquibase). At startup, Spring automatically runs these scripts in order, applying only the changes that haven't been recorded in the database's internal history table.

Q.41. What transaction management capabilities does Spring Boot provide?

A: Spring Boot provides robust **declarative transaction management** capabilities using the `@Transactional` annotation. This ensures **ACID properties** (Atomicity, Consistency, Isolation, Durability) for database operations. It automatically starts a transaction when a method is called and ensures that all operations commit together or roll back completely if an error occurs.

Q.42. Can transactions be managed externally (DB, message broker), or must they always be within the JVM?

A: Transactions can be managed externally, especially when dealing with distributed services. Standard JVM transactions typically rely on a single resource (like a database). However, for complex scenarios involving a database and a message broker, patterns like the **Transactional Outbox Pattern** ensure that both the database update and message sending happen atomically, even without a heavy distributed transaction manager (2PC).

Q.43. How would you design transaction boundaries in an e-commerce checkout flow?

A: I would define a single, encompassing transaction boundary at the **Service layer method** responsible for finalizing the order (e.g., `checkoutService.placeOrder()`). This transaction must atomically handle critical steps: **1)** Decrementing inventory stock, and **2)** Creating the final order record. If either step fails, the entire transaction must roll back to maintain consistency.

Q.44. How do you configure multiple data sources in a Spring Boot application?

A: You configure multiple data sources by defining separate property prefixes for each database (e.g., `spring.datasource.db1` and `spring.datasource.db2`). You then create custom **@Configuration** classes to manually define and configure a separate **DataSource** bean, **EntityManagerFactory** bean, and **PlatformTransactionManager** bean for each database.

Q.45. Explain the complete flow to connect two or more separate databases.

A: The flow requires defining separate configuration classes for each database. Each class must define three unique beans: the **DataSource** (connection details), the **LocalContainerEntityManagerFactoryBean** (maps entities to that database), and the **JpaTransactionManager** (manages transactions). Repositories and service methods then use the **@Qualifier** annotation to specify which transaction manager or repository to use.

Q.46. How do you implement pagination efficiently using Spring Data JPA?

A: Spring Data JPA makes pagination efficient through the **PagingAndSortingRepository** interface, which **JpaRepository** extends. You implement pagination by creating a **Pageable** object (specifying page number and size). You then call the repository method

findAll(pageable), which returns a **Page** object containing the chunk of results and metadata like the total count.

Error Handling & Exceptions (Advanced)

Q.47. How do you implement global exception handling with proper API responses?

A: You implement global exception handling by creating a class annotated with **@ControllerAdvice** (often called a Global Exception Handler). Inside this class, you define methods annotated with **@ExceptionHandler** that catch specific exception types. These methods return a **ResponseEntity** object, allowing you to set the exact HTTP status code (e.g., 400 or 500) and customize the JSON error payload.

Q.48. What are best practices for designing a reusable exception-handling layer?

A: Best practices include using **@ControllerAdvice** to centralize all handling logic. You should define a standard **Error Response DTO** (Data Transfer Object) that all handlers return, containing fields like a timestamp, clear message, and error code. Crucially, log the full stack trace on the server side, but never expose it to the client.

Q.49. How do you handle 404 errors and customize error payloads?

A: You handle 404 (Not Found) errors by using an **@ExceptionHandler** method specifically for **NoHandlerFoundException** or custom exceptions like **ResourceNotFoundException**. To customize the payload, the handler method returns a **ResponseEntity** with the **HttpStatus.NOT_FOUND** status and a customized JSON body, instead of the default Spring error page.

Q.50. How do you fix the Whitelabel Error Page in Spring Boot?

A: The Whitelabel Error Page is the default fallback HTML page shown when an error occurs and no specific handler is found. You fix it by providing a more user-friendly page (by creating an HTML file named **error.html** in your **resources/templates** directory). The proper way, however, is to use **@ControllerAdvice** and **@ExceptionHandler** to handle errors programmatically and return a JSON response.

REST Architecture & API Design

Q.51. What is API versioning and what strategies exist to implement versioning?

A: **API versioning** is the practice of managing changes to your API over time while maintaining stability and functionality for existing clients. The main strategies are **URI versioning** (e.g., /api/v1/users), which is simple and explicit. Other methods include **Query Parameter versioning** (e.g., /api/users?v=1), or **Header versioning** (where the client sends a custom HTTP header requesting the version).

Q.52. What are REST API best practices in enterprise systems?

A: Best practices include using **noun-based URLs** (e.g., /users, not /getUsers), using the correct **HTTP verbs** (GET, POST, PUT, DELETE) for actions, and ensuring the API is **stateless** (no client session data stored on the server). All responses should consistently use standard HTTP status codes and return data in a standard format, typically JSON.

Q.53. When do you return ResponseEntity vs a direct body?

A: You should return a **direct body** (e.g., a simple DTO) when you are confident the operation will succeed and you only need the default 200 OK status. You must return a **ResponseEntity** when you need control over the HTTP response details, such as setting a custom **status code** (e.g., 201 Created, 404 Not Found), or adding specific HTTP headers.

Q.54. How do you design a complete REST API (status codes, headers, versioning, standards)?

A: I design a complete REST API by focusing on **resources** and **HTTP verbs**. I would use **URI versioning** (e.g., /v1). For standards, I use **JSON** for the body and rely on standard **HTTP status codes** for feedback (e.g., 200 for OK, 201 for Created, 400 for Bad Request). I also ensure necessary headers, like Content-Type: application/json, are consistently set.

Logging, Observability & Events

Q.55. How do you implement structured logging using SLF4J / Logback / MDC?

A: You implement structured logging by configuring **Logback** to output in JSON format, rather than plain text, using a JSON appender. You utilize **SLF4J's** formatting methods to pass key-value pairs directly into the log statement. The **MDC (Mapped Diagnostic Context)** is used to attach contextual data, like a unique traceId or userId, to all logs produced by the current thread, making logs easy to search and analyze.

Q.56. How would you integrate Sleuth or Micrometer Tracing?

A: You integrate tracing by adding the appropriate Spring Boot starter dependency (e.g., spring-boot-starter-actuator and the tracing binder). This automatically generates and attaches unique **Trace IDs** and **Span IDs** to every request entering the application. These IDs are then propagated to downstream services via HTTP headers, allowing you to trace a request's entire journey across multiple microservices.

Q.57. How do you use application events to decouple components inside a Spring Boot app?

A: You use application events to allow components to communicate **asynchronously** without knowing each other exist. One component publishes an event (e.g., UserCreatedEvent) by calling **ApplicationEventPublisher.publishEvent()**. Another component, the listener, uses the **@EventListener** annotation to subscribe to that specific event type. This completely decouples the publisher from the subscriber, improving modularity.

Deployment & Packaging

Q.58. What is the difference between embedded server deployment vs external server deployment?

A: **Embedded server deployment** means the application (JAR file) includes the web server (like Tomcat) inside itself, making it self-contained and runnable with a simple `java -jar` command. **External server deployment** requires you to package the app as a WAR file and deploy it onto a separately installed and managed external application server.

Q.59. What are the benefits and drawbacks of embedded servers?

A: The benefits are **simplicity** and **portability**; the app is entirely self-contained and easy to deploy (just one JAR). The drawbacks are that the application size is larger because it includes the server, and you lose centralized management features often provided by external servers.

Q.60. Steps to migrate an embedded Tomcat app to an external Tomcat server.

A: First, change the packaging type in your pom.xml from jar to **war**. Then, you need to modify the main application class to extend **SpringBootServletInitializer**. You must also mark the embedded Tomcat dependency as `<scope>provided</scope>` so it is ignored during packaging but available on the external server.

Q.61. What is your deployment strategy for AWS vs Azure?

A: For **AWS**, my strategy would focus on containerization using **ECS** (Elastic Container Service) or **EKS** (Kubernetes), coupled with managed services like RDS for the database. For **Azure**, I would use **Azure App Service** or **Azure Kubernetes Service (AKS)**, leveraging Azure's built-in platform features for streamlined deployment.

Q.62. How do you configure environment properties for cloud deployments?

A: I configure properties using **environment variables** or a managed **Secret Store** (like AWS Secrets Manager or Azure Key Vault). The cloud platform loads these variables at runtime, and Spring Boot automatically binds them to the configuration, overriding any values set locally in application.properties.

Q.63. How does Spring Boot simplify creating Docker images?

A: Spring Boot simplifies image creation by creating a single executable JAR that includes the server, removing the complex setup steps required by traditional applications. Newer Spring Boot versions also include **Cloud Native Buildpacks**, which allow you to create optimized Docker images without writing a single Dockerfile.

Q.64. Explain the full process of writing a Dockerfile for a Spring Boot app.

A: First, you define a base image (like a minimal JRE). You copy the executable JAR file into the container. You set environment variables (like JVM memory limits). Finally, you define the

entry point using the **ENTRYPOINT** ["java", "-jar", "app.jar"] command to run the application when the container starts.

Q.65. What docker commands do you use to build, tag & push images?

A: I use **docker build -t app-name:latest** . to build and tag the image from the Dockerfile. I use **docker tag app-name:latest registry/app-name:v1** to apply a specific tag for version control. I use **docker push registry/app-name:v1** to send the final, tagged image to a remote container registry.

Q.66. How do you manage environment-specific configs in Dockerized apps?

A: I use **Spring Profiles** to manage config files (e.g., application-prod.yml). I use **environment variables** passed into the container at runtime (e.g., in Kubernetes or docker run -e). These environment variables set the **SPRING_PROFILES_ACTIVE** flag and pass in sensitive secrets, which Spring then consumes.

ASYNC, Queues, and Reactive

Q.67. How does Spring Boot support event-driven architecture?

A: Spring Boot supports event-driven architecture primarily through **Spring Application Events** and integration with external message brokers. For internal decoupling, you use the **ApplicationEventPublisher** to publish events and methods annotated with **@EventListener** to subscribe to them. For external communication, Spring provides auto-configuration for connecting to queues like Kafka or RabbitMQ.

Q.68. How do you integrate Spring Boot with a message queue (Kafka, RabbitMQ)?

A: You integrate by including the necessary Spring Starter dependency (e.g., spring-kafka or spring-rabbit). Spring Boot auto-configures the connection, serializers, and templates. You then create a **producer** component that uses the corresponding template to send messages to the queue, and a **consumer** component that listens for incoming messages using the **@KafkaListener** or **@RabbitListener** annotation.

Q.69. How do you handle async tasks using @Async and Executor pools?

A: You enable asynchronous processing by using the **@EnableAsync** annotation on a configuration class and marking individual methods with **@Async**. Spring automatically executes these methods in a separate thread, usually drawn from the default thread pool. For production, you customize this behavior by defining your own **Executor** bean to control thread pool size and behavior.

Q.70. How would you process heavy files asynchronously?

A: I would process heavy files asynchronously by first saving the file to persistent storage (like S3 or a local file system). Then, I would create an **@Async** service method to handle the parsing and processing. Alternatively, I would push a **message** containing the file location/ID onto a queue, and a separate worker service would pick up the message and perform the heavy processing.

Q.71. How does Spring Boot handle backpressure in reactive streams?

A: Spring Boot relies on the underlying reactive libraries (like Project Reactor) to handle **backpressure**. Backpressure is a mechanism where a slow consumer tells a fast producer to slow down the flow of data. The consumer indicates how many items it is ready to handle, ensuring the system doesn't crash when overwhelmed by a high volume of events.

Q.72. What reactive programming support does Spring Boot offer?

A: Spring Boot offers full reactive programming support through **Spring WebFlux** (the reactive web framework) and integrations with Project Reactor. This includes the **WebClient** for non-blocking HTTP calls and support for reactive data access (Spring Data R2DBC). These tools allow developers to build non-blocking, high-concurrency applications.

Q.73. How would you design integration tests for a Spring Boot app interacting with external APIs?

A: I would design the tests to use a dedicated mock server, like **WireMock**, instead of simple mocking frameworks. WireMock runs a real, local HTTP server that fully simulates the external API. This allows the application's REST client (like WebClient or RestTemplate) to make a genuine network call, testing serialization, HTTP status codes, and network error handling close to reality.

Q.74. What testing strategies do you follow for distributed systems?

A: I rely on the **Testing Pyramid** approach, shifting focus away from slow End-to-End (E2E) tests. The strategy focuses on: **1) Unit/Component Tests** (testing each service in isolation with mocked dependencies like the database). **2) Contract Tests** (using tools like Pact) to verify that the APIs between services adhere to agreed-upon specifications. **3) E2E Tests** are only used for critical, high-level business flows.

Microservices Architecture & Operations

Q.77. How do you implement rate limiting in a Spring Boot microservice?

A: I would implement rate limiting using the Token Bucket Algorithm. This involves using a central store like Redis to track how many requests (tokens) a client has remaining within a time window. Before processing a request, a custom Spring filter checks Redis; if the client's bucket is empty, the request is rejected immediately with a 429 Too Many Requests status.

Q.78. How do you detect and fix memory leaks in a Spring Boot app?

A: I detect leaks by monitoring heap usage and taking a Heap Dump when memory is high. I analyze the dump using tools like Eclipse Memory Analyzer (MAT) to find objects that are still referenced but should have been garbage collected. Fixes usually involve removing unintentional references, often large static collections or unclosed resources that are preventing objects from being released.

Q.79. How do you integrate centralized logging (ELK / Loki / CloudWatch)?

A: I integrate centralized logging by configuring the Spring Boot application to use a structured JSON format for its output. Then, I use a log shipper agent (like Filebeat or Fluentd) running alongside the application to tail the log files. This agent automatically sends the structured log data to the central aggregation platform (ELK/Loki) for indexing and analysis.

Q.80. How do you use configuration servers (Spring Cloud Config) for distributed microservices?

A: Spring Cloud Config Server is used to store all configuration files (properties, YAML) externally, typically in a Git repository. Microservices connect to the Config Server at startup, fetching their required configuration based on their name and active profile. This allows

central management of secrets and properties without restarting services when simple config changes occur.

Q.81. How do you design a distributed tracing strategy?

A: A distributed tracing strategy is designed by injecting unique identifiers into every request. I use Micrometer Tracing (formerly Sleuth) to automatically generate a unique Trace ID and Span ID when a request enters the system. This ID is propagated via HTTP headers to all downstream services, allowing monitoring tools (like Zipkin or Jaeger) to visualize the request's journey and pinpoint latency.

Q.82. How do you use API Gateways with Spring Boot (routing, filters, auth)?

A: An API Gateway (like Spring Cloud Gateway) acts as the single entry point for all client requests. It performs Routing (forwarding requests to the correct service), uses Filters for cross-cutting concerns like logging and rate limiting, and centralizes Authentication by validating JWTs before passing the request to the services.

Q.83. How do you decide between mono-repo vs multi-repo for microservices using Spring Boot?

A: I favor a multi-repo approach when the services are highly independent, managed by different teams, and require distinct deployment pipelines. I choose a mono-repo when the services share significant amounts of core code, are managed by a single team, or require frequent, synchronized changes across multiple services (like shared domain models).